

Discovery Executive Summary

Project: discovery-15july-01 · Generated: 15/07/2026, 20:00:54

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 7 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—
2	Code Quality & Complexity Analysis	HIGH RISK	44 / 100 — Moderate
3	Frontend Modernization Analysis	HIGH RISK	—
4	Backend Modernization Analysis	HIGH RISK	—
5	Testing & Quality Assurance Analysis	HIGH RISK	—
6	Security Analysis	HIGH RISK	—
7	Technical Debt	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service Layer (H2), Missing Repository Pattern (H3), Domain Boundary Violations (H8), Shared Database Coupling (H9), and Parallel API Runtimes (H10).

EXECUTIVE SUMMARY

Analysis covered **backend** (15 PHP application files: 6 API controllers, 4 Eloquent models, 2 injectable services, 0 repositories), **frontend** (15 TS/TSX/JSX source files across pages, components, hooks, and store), and **dev-api** (6 JS files mirroring 18 Laravel routes) from `shende-shweta/FSDKC@main` via GitHub REST API (recursive tree + raw content fetch). The Klearcom monorepo runs three parallel runtimes with no repository layer and thin service coverage. Backend layering is the dominant risk: **25 controller-to-model Eloquent access points** bypass application services, **83**

persistence access points occur outside any repository abstraction (Eloquent + MongoDB/in-memory store delegations across Laravel and dev-api), and **12 cross-domain query sites** in `DashboardController` and `LegacyReportController` couple Discovery and Connect models without an anti-corruption layer. Four of five MariaDB tables are accessed from three or more modules (**80% shared-table coupling**), and Laravel + `dev-api` duplicate reachability KPI math, IVR `buildTree`, and dashboard aggregation in parallel. Frontend architecture is comparatively healthy (avg **86 LOC** per view component, centralized `api/client.ts`, max prop-drilling depth 2), but one legacy class component lacks lifecycle cleanup. Overall verdict: **High Risk**, driven by H2, H3, H8, H9, and H10.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	74 LOC avg	GOOD
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	25 access points	HIGH RISK
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	83 access points	HIGH RISK
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	4 (<code>extract</code> sites)	MODERATE
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100% ORM	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (max 276 LOC)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	12	HIGH RISK
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	80% (4/5 tables)	HIGH RISK
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	86 LOC avg	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	0 inline fetch/axios	GOOD

F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	2 levels	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	1 class component	MODERATE
H10	Parallel API Runtimes (additional)	Duplicate API implementations across runtimes	0	1	≥2	2 (Laravel + dev-api)	HIGH RISK

1.4 Actions Required

Hotspot	Action	Rating	Priority
H2 — Missing Service Layer	Introduce <code>ConnectMonitorService</code> , <code>DiscoveryJobService</code> , and <code>DashboardKpiService</code> ; move all 25 controller model-access points into application services with constructor DI.	HIGH RISK	CRITICAL
H3 — Missing Repository Pattern	Create repository interfaces for all 4 Eloquent models and 3 MongoDB collections; route 83 persistence access points through injected repository implementations.	HIGH RISK	CRITICAL
H5 — Shared Utility Abuse	Replace <code>extract(\$filters)</code> with typed DTO mappers; extract shared <code>IvrTreeBuilder</code> from duplicated <code>buildTree</code> in <code>LegacyReportController</code> , <code>DiscoveryController</code> , and <code>dev-api/src/store.js</code> .	MODERATE	MEDIUM
H8 — Domain Boundary Violations	Split cross-domain access in <code>DashboardController</code> and <code>LegacyReportController</code> ; enforce Discovery/Connect bounded contexts with published read APIs.	HIGH RISK	CRITICAL
H9 — Shared Database Coupling	Assign table ownership per domain in <code>docker/mariadb/init.sql</code> ; introduce per-domain migration files and integration views for cross-domain KPIs.	HIGH RISK	CRITICAL
H10 — Parallel API Runtimes	Deprecate <code>dev-api</code> duplicate runtime; consolidate to Laravel API with Docker Compose for local dev, or generate dev-api from OpenAPI spec.	HIGH RISK	CRITICAL
F5 — Legacy Component Patterns	Rewrite <code>LegacyMonitorPoller.jsx</code> as function component with <code>useEffect</code> cleanup; add Error Boundary around <code>LegacyDashboardWidget</code> .	MODERATE	MEDIUM

1.5 Expected Outcomes

- **Testable business logic:** Application services and repositories enable unit tests without HTTP or database bootstrapping, cutting test setup time for Connect reachability and Discovery tree workflows.

- **Independent domain evolution:** Bounded contexts with owned schemas let Discovery and Connect teams ship schema changes without cross-module regressions in dashboard KPIs.
- **Single source of truth:** Eliminating the parallel `dev-api` runtime removes behavioral drift between local development and production Laravel deployments.
- **Reduced change amplification:** Centralizing reachability calculation and IVR tree building in one service eliminates the current four-way duplication across controllers, services, and Node handlers.
- **Frontend stability:** Migrating the legacy class component and adding Error Boundaries prevents interval leaks and uncaught render errors during SPA navigation.

Full report with §1.2 evidence and §1.3 Mermaid diagrams: `target/docs/discovery/01-architecture-design.md` (PDF conversion runs automatically in the orchestration UI).

2. Code Quality & Complexity Analysis

OVERALL CODEBASE RATING — CODE QUALITY & COMPLEXITY

High Risk

Driven by High-Risk cyclomatic complexity in frontend page components (H1), oversized `ConnectPage` function (H3), cross-runtime business-logic duplication (H4), parallel Laravel/dev-api workflow copies (H9), and PHP `extract()` dynamic variables (H11).

EXECUTIVE SUMMARY

Re-analysis covered **backend** (16 PHP application files, 807 SLOC), **frontend** (14 TS/TSX/JSX files, 874 SLOC), and **dev-api** (6 JS files, 720 SLOC) — **36 files / 2,401 SLOC** total — from `shende-shweta/FSDKC@main` via GitHub REST API (tree + raw content; Commits API partially rate-limited). No cyclomatic-complexity linter (`eslint-plugin-complexity`, `phpmd`, Sonar) is configured; metrics were derived by automated branch/loop counting on fetched source. The dominant risks remain **frontend page-level complexity** (`ConnectPage.tsx` cyclomatic complexity **37**, **201 LOC** default-export function), **cross-runtime business-logic duplication** (~**13.2%** of codebase duplicated between Laravel and `dev-api` for KPI aggregation, realtime test orchestration, and IVR `buildTree`), and **three `extract()` dynamic-variable sites** in legacy PHP code. No files exceed 1,000 LOC (largest: `dev-api/src/server.js` at **224 SLOC**). Git history is shallow (**3 commits**, single author `ksabai-gl` since June 2026), so churn and ownership signals are healthy but low-confidence. Overall verdict: **High Risk**, driven by H1, H3, H4, H9, and H11.

2.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	---------	-------------	------	----------	-----------	----------	--------

H1	High Cyclomatic Complexity	Max complexity per method	<10	10–20	>20	37 (<code>ConnectPage</code>)	HIGH RISK
H2	Large Classes	Largest class LOC	<300	300–1000	>1000	224 LOC (<code>dev-api/src/server.js</code>)	GOOD
H3	Large Functions	Largest function LOC	<50	50–200	>200	201 LOC (<code>ConnectPage</code>)	HIGH RISK
H4	Business Logic Duplication	Duplicated business logic %	<5%	5–10%	>10%	13.2% (317 / 2,401 LOC)	HIGH RISK
H5	Duplicate Code (general)	Overall duplicate code %	<5%	5–10%	>10%	8.6% (206 / 2,401 LOC)	MODERATE
H6	High Churn Areas	Monthly changes (top files)	<5	5–10	>10	2 changes/mo (top app files)	GOOD
H7	Defect-Prone Files	Fix commits (hottest file)	1–3	4–5	>5	1 fix commit (<code>frontend/src/App.tsx</code>)	GOOD
H8	Ownership Issues	Top-author ownership %	>80%	60–80%	<60%	100% (1 author / 3 commits)	GOOD
H9	Parallel Runtime Duplication (additional)	Duplicated workflow LOC across Laravel + dev-api / backend LOC	<5%	5–15%	>15%	20.8% (317 / 1,527 backend+dev-api LOC)	HIGH RISK
H10	God Page Components (additional)	Largest page component LOC (UI + data + realtime)	<150	150–300	>300	208 LOC (<code>ConnectPage.tsx</code>)	MODERATE
H11	PHP extract() Dynamic Variables (additional)	<code>extract()</code> call sites in application code	0	1–2	>2	3 (<code>LegacyReportController</code> , <code>LegacyDataMapper</code> ×2)	HIGH RISK

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	25%	76	19.0
Code Churn	25%	10	2.5
Defect Density	20%	15	3.0

Class/Function Size	15%	75	11.25
Business Logic Duplication	10%	78	7.8
Developer Ownership Risk	5%	5	0.25
Hotspot Score	100%		44 / 100

2.5 Actions Required

Hotspot	Action	Rating	Priority
H1 — High Cyclomatic Complexity	Extract <code>useModulePageQueries</code> hook; split <code>ConnectPage</code> and <code>DiscoveryPage</code> into ≤ 80 LOC sub-components; enable ESLint complexity rule (max 15) on <code>frontend/src/pages/</code> .	HIGH RISK	CRITICAL
H3 — Large Functions	Decompose <code>ConnectPage</code> (201 LOC) into <code>useConnectPageState.ts</code> + 3 presentational components; apply same pattern to <code>DiscoveryPage</code> (154 LOC).	HIGH RISK	HIGH
H4 — Business Logic Duplication	Create <code>DashboardKpiCalculator</code> and shared test-step definitions in Laravel; remove duplicated KPI and realtime blocks from <code>dev-api/src/server.js</code> and <code>dev-api/src/realtime.js</code> .	HIGH RISK	CRITICAL
H5 — Duplicate Code	Extract <code>TreeBuilder</code> utility from four <code>buildTree</code> copies; add <code>jscpd</code> / <code>phpcpd</code> CI gate at 5% threshold.	MODERATE	MEDIUM
H9 — Parallel Runtime Duplication	Deprecate <code>dev-api</code> runtime or auto-generate from Laravel OpenAPI; add contract tests until removal.	HIGH RISK	CRITICAL
H10 — God Page Components	Introduce <code>ModulePageLayout</code> , <code>EntityForm</code> , and <code>TestSessionPanel</code> shared components; cap page files at 120 LOC.	MODERATE	MEDIUM
H11 — PHP <code>extract()</code> Dynamic Variables	Replace <code>extract(\$filters)</code> with typed <code>CarrierSummaryFilter</code> DTO; refactor <code>LegacyDataMapper</code> to explicit key access; add PHPStan <code>extract()</code> ban.	HIGH RISK	HIGH

2.6 Expected Outcomes

- **Lower defect rate on UI changes:** Splitting `ConnectPage` / `DiscoveryPage` and extracting shared hooks reduces cyclomatic complexity from 37 to a testable target of <15 per function.
- **Single source of truth for KPIs and test pipelines:** Consolidating dashboard and realtime logic in Laravel eliminates the ~13% duplicated business-rule surface between production and `dev-api`.
- **Safer refactors:** Extracting `TreeBuilder` and domain services lets IVR tree and reachability changes propagate from one module instead of four `buildTree` copies.
- **Eliminated dynamic-variable risk:** Replacing `extract()` with typed DTOs enables PHPStan static analysis and prevents variable-injection bugs in legacy report endpoints.
- **Faster reviews:** Smaller page components and a complexity lint gate keep new features from re-expanding god components.

3. Frontend Modernization Analysis

OVERALL CODEBASE RATING — FRONTEND MODERNIZATION

High Risk

Driven by High-Risk UI Component Duplication (H1), Global State Dependencies (H4), Inline Styles Without Design Tokens (H6), and Missing Accessibility Roles (H7).

EXECUTIVE SUMMARY

Analysis covered **14 frontend source files** (8 view components across `pages/` and `components/`) from `shende-shweta/FSDKC@main` via GitHub REST API (tree + raw content fetch). The Klearcom SPA is predominantly modern — function components, TanStack React Query v5, Zustand v5, and React Router v7 — with a centralized `api/client.ts` and reusable `useRealtimeTest` hook. However, `ConnectPage.tsx` and `DiscoveryPage.tsx` duplicate an identical workbench shell (entity form, live feed, two-column grid, transcript panel), inflating UI duplication to **25%**. Six of eight view components depend on shared global state (Zustand, React Query cache, or the singleton `api` client), and **30 inline `style={{}}` occurrences** bypass the CSS token system in `index.css`. One legacy class component (`LegacyMonitorPoller.jsx`) lacks `componentWillUnmount` cleanup and leaks intervals; two orphan legacy modules coexist with the React Query stack. Zero `aria-*` / `role` attributes were found across all view components. Overall verdict: **High Risk**, driven by H1 (UI duplication), H4 (global state coupling), H6 (inline styles), and H7 (missing accessibility).

3.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	25.0% (2/8)	HIGH RISK
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	87.5% (7/8)	MODERATE
H3	Massive Components	Largest component LOC	<200	200–500	>500	222 (<code>ConnectPage.tsx</code>)	MODERATE
H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	75.0% (6/8)	HIGH RISK

H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	2 levels	GOOD
H6	Inline Styles / No Design Tokens (additional)	Total <code>style={{}}</code> occurrences (target <10)	<10	10–20	>20	30 across 7 files	HIGH RISK
H7	Missing Accessibility Roles (additional)	Components with <code>aria-*</code> or <code>role</code> (target >80%)	>80%	50–80%	<50%	0% (0/8)	HIGH RISK

3.5 Actions Required

Hotspot	Action	Rating	Priority
H1 — UI Component Duplication	Extract <code>DomainWorkbenchPage</code> , <code>EntityFormCard</code> , and <code>TranscriptPanel</code> from <code>ConnectPage.tsx</code> and <code>DiscoveryPage.tsx</code> ; reduce duplicate shell to domain hooks only.	HIGH RISK	CRITICAL
H2 — Legacy Class-Based Components	Convert <code>LegacyMonitorPoller.jsx</code> to <code>useMonitorReachability</code> hook; migrate <code>LegacyDashboardWidget.tsx</code> to React Query or delete both orphan modules.	MODERATE	MEDIUM
H3 — Massive Components	Split <code>ConnectPage.tsx</code> (222 LOC) and <code>DiscoveryPage.tsx</code> (176 LOC) into container hooks + presentational sub-components; target <150 LOC per page.	MODERATE	MEDIUM
H4 — Global State Dependencies	Replace <code>useUiStore</code> selection with URL search params; add <code>frontend/src/api/queryKeys.ts</code> ; scope query invalidation to domain keys.	HIGH RISK	HIGH
H6 — Inline Styles	Add CSS utility classes to <code>index.css</code> ; migrate 30 inline <code>style={{}}</code> occurrences (18 in domain pages) to token-based classes.	HIGH RISK	HIGH
H7 — Missing Accessibility	Add <code>aria-live</code> to <code>LiveTestFeed</code> , keyboard-accessible row selection in domain tables, and <code>htmlFor</code> on form labels.	HIGH RISK	HIGH

3.6 Expected Outcomes

- **Shared component library** eliminates the 25% duplicate page-shell pattern; Connect and Discovery fixes ship once via `DomainWorkbenchPage` and `TranscriptPanel`.
- **Hooks-first migration** of `LegacyMonitorPoller` removes interval leaks and aligns all data access on React Query with testable custom hooks.
- **Scoped state** (URL params + domain query keys) reduces cross-feature invalidation fan-out and makes monitor/job selection shareable via deep links.

- **Design-token consistency** from replacing 30 inline styles lowers theming cost and aligns with the existing `:root` token system in `index.css`.
- **Accessibility baseline** (`aria-live`, keyboard rows, label associations) makes realtime test feeds and data tables operable for keyboard and screen-reader users in an operations-focused product.

Deliverables saved:

- Full report: `target/docs/discovery/03-frontend-modernization.md`
- Pipeline summary: `agent-runs/20260715T193441_nth87z/03-frontend-modernization-summary.md`

4. Backend Modernization Analysis

OVERALL CODEBASE RATING — BACKEND MODERNIZATION

High Risk

Driven by Direct SQL/ORM Outside Data Layer (H3), Missing Service Layer (H5), API Sprawl (H6), Missing API Governance (H7), and Parallel API Runtimes (H8).

EXECUTIVE SUMMARY

Analysis covered **24 controllers/handlers** (6 Laravel API controllers + 18 Express route handlers), **37 REST endpoints**, and **2 injectable service classes** (0 repositories) from `shende-shweta/FSDKC@main` via GitHub REST API (recursive tree + raw content fetch). The Klearcom backend runs two parallel API runtimes: Laravel 12 uses constructor injection for `MongoService` and `RealTimeTestService`, but **25 of 32 Eloquent access points (78%)** remain in controllers with no repository layer, and four controllers embed KPI math, IVR tree building, and reachability calculations inline. Three PHP `extract()` calls — including one on raw `$request->all()` — create untyped variable scope from user input. The Node `dev-api` holds all relational state in a module-level mutable `store` singleton with **18 inline route handlers** mirroring Laravel. No OpenAPI spec, API versioning, or contract tests exist; CI runs PHPUnit and frontend build only. Overall verdict: **High Risk**, driven by data-layer bypass (H3), missing service tier across dual runtimes (H5), API sprawl and zero governance (H6–H7), and parallel API implementations (H8).

4.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Dynamic Variable Creation	Dynamic-var-from-input occurrences	0	1–10	>10	3 (<code>extract()</code> calls)	MODERATE

H2	Global Mutable State	Globals / mutable static state	0	1–5	>5	2 module-level stores	MODERATE
H3	Direct SQL Outside Data Layer	Data-layer compliance %	>90%	60–90%	<60%	22% (7/32 Eloquent in services)	HIGH RISK
H4	Static / Singleton Abuse	Business-logic static/singleton classes	0	1–5	>5	0	GOOD
H5	Missing Service Layer	Handlers with inline business logic	<10	10–20	>20	22 (4 Laravel controllers + 18 dev-api routes)	HIGH RISK
H6	API Sprawl	Documented & governed endpoints %	>90%	80–90%	<80%	15% single-source (3/20 capabilities); 0% governed	HIGH RISK
H7	Missing API Governance	Governance compliance %	100%	90–99%	<90%	0% (no spec, versioning, or contract tests)	HIGH RISK
H8	Parallel API Runtimes (additional)	Capabilities with duplicate Laravel + dev-api handlers (target 0)	0	1–5	>5	17 duplicated capabilities	HIGH RISK
H9	Missing Input Validation on dev-api (additional)	POST/PUT routes without schema validation (target 0)	0	1–3	>3	4 unvalidated write routes	HIGH RISK

4.5 Actions Required

Hotspot	Action	Rating	Priority
H1 — Dynamic Variable Creation	Replace <code>extract(\$filters)</code> in <code>LegacyReportController</code> with validated request DTO; refactor <code>LegacyDataMapper</code> to explicit field access; add CI ban on <code>extract()</code> .	MODERATE	HIGH
H2 — Global Mutable State	Encapsulate <code>dev-api/src/store.js</code> in an injectable repository; eliminate module-level business mutation or retire dev-api.	MODERATE	MEDIUM
H3 — Direct SQL Outside Data Layer	Create Eloquent repositories for all four models; move 25 controller ORM calls into repositories; target >90% data-layer compliance.	HIGH RISK	CRITICAL
H5 — Missing Service Layer	Add <code>DashboardService</code> , <code>DiscoveryService</code> , <code>ConnectService</code> ; move KPI/tree/reachability logic out of controllers and dev-api routes.	HIGH RISK	CRITICAL
H6 — API Sprawl	Consolidate 17 duplicated capabilities under Laravel; deprecate overlapping dev-api routes; publish canonical endpoint list.	HIGH RISK	HIGH
H7 — Missing API Governance	Author <code>docs/openapi.yaml</code> ; add <code>/api/v1/</code> versioning; introduce contract tests in CI via schemathesis or equivalent.	HIGH RISK	HIGH

H8 — Parallel API Runtimes	Retire or proxy <code>dev-api</code> to Laravel; eliminate triplicated reachability and dashboard KPI logic.	HIGH RISK	CRITICAL
H9 — Missing Input Validation on dev-api	Add express-validator schemas on all POST routes; remove or secure <code>bulk-import</code> endpoint.	HIGH RISK	HIGH

4.6 Expected Outcomes

- **Repository layer** moves 25 controller Eloquent calls behind testable boundaries, raising data-layer compliance from 22% to >90% and enabling mocked persistence in PHPUnit feature tests.
- **Application services** (`DashboardService` , `DiscoveryService` , `ConnectService`) eliminate triplicated KPI, tree, and reachability logic — fixes ship once and propagate to all entry points.
- **Retiring parallel dev-api routes** removes 17 duplicate handlers and the module-level `store` singleton, halving API surface area and integration drift risk.
- **OpenAPI spec + contract tests** catch breaking response-shape changes before merge, giving the React SPA a machine-verifiable integration contract.
- **Replacing `extract()` with typed DTOs** closes the variable-scope injection vector in legacy reporting and aligns with AGENTS.md engineering standards.

5. Testing & Quality Assurance Analysis

OVERALL CODEBASE RATING — TESTING & QUALITY ASSURANCE

High Risk

Driven by High-Risk untested critical logic (H1), low coverage (H2), missing integration tests (H3), missing contract tests (H4), absent E2E tests (H7), and assertion-free unit tests (H8).

EXECUTIVE SUMMARY

Test-suite health across the FSDKC monorepo is **critically thin**. The backend ships **PHPUnit 11** with only **2 unit test files** (3 test methods) that assert hard-coded arrays and random numbers — none import `App\` production classes. The React 19 / Vite 6 / TypeScript frontend (**15 source files**) and Node `dev-api` (**6 source files, 18 route handlers**) have **zero** test files and no Jest, Vitest, Cypress, or Playwright configuration. Estimated overall coverage by test-file-to-source ratio is **~6%** (2 test files / 36 application source files); per-layer split is **backend ~13% file ratio (0% effective)**, **frontend 0%**, and **dev-api 0%**. No coverage reports (`lcov`, `clover.xml`) exist in the repository. GitHub Actions runs `vendor/bin/phpunit` against MariaDB on every PR (backend gate), but the frontend job only runs `npm run build` and `dev-api` is excluded entirely. The highest-risk gaps are untested Discovery/Connect orchestration (`RealTimeTestService`, six API controllers), no integration or contract tests for **19 Laravel API routes** (37 total across dual runtimes), and no E2E coverage for realtime SSE workflows. Overall verdict: **High Risk**.

5.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Untested Critical Logic	Critical modules with zero tests	0	1–3	>3	9 modules	HIGH RISK
H2	Low Test Coverage	Overall coverage %	>80%	50–80%	<50%	~6% estimated (2/36 app source files; no coverage report)	HIGH RISK
H3	Missing Integration Tests	Boundaries covered %	>70%	30–70%	<30%	0% (0 Feature/HTTP/DB tests)	HIGH RISK
H4	Missing Contract Tests	APIs with contract tests %	>80%	40–80%	<40%	0% (0/19 Laravel endpoints; 0/18 dev-api routes)	HIGH RISK
H5	Flaky / Skipped Tests	Skipped/flaky test count	0	1–5	>5	0	GOOD
H6	No CI Test Gate	Tests enforced in CI	Required gate	Runs, not required	No CI test run	Backend PHPUnit on PR; frontend build-only; dev-api excluded	MODERATE
H7	No End-to-End Tests (additional)	Critical user flows with E2E specs	≥2 flows	1 flow	0 flows	0 E2E specs	HIGH RISK
H8	Assertion-Free Unit Tests (additional)	Unit tests exercising production code %	>80%	40–80%	<40%	0% (0/2 tests import <code>App\</code> code)	HIGH RISK

5.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — Untested Critical Logic	Add PHPUnit tests for <code>RealTimeTestService</code> , all six API controllers, and <code>MongoService</code> ; add Vitest tests for <code>useRealtimeTest</code> , pages, and <code>api/client.ts</code> ; add or remove dev-api with parity tests.	HIGH RISK	CRITICAL
H2 — Low Test Coverage	Establish coverage baselines (PHPUnit <code>--coverage-text</code> , Vitest <code>--coverage</code>); enforce 75% backend / 60% frontend thresholds in CI before refactors.	HIGH RISK	CRITICAL
H3 — Missing Integration Tests	Create <code>backend/tests/Feature/</code> with <code>RefreshDatabase</code> + MariaDB; test Discovery job lifecycle, Connect check flow, and SSE streaming boundaries.	HIGH RISK	HIGH

H4 — Missing Contract Tests	Publish OpenAPI spec; add JSON Schema contract tests for all 19 API endpoints; validate TypeScript types against fixtures.	HIGH RISK	HIGH
H6 — No CI Test Gate	Add <code>vitest run</code> to frontend CI job; add dev-api test job or deprecate runtime; mark all test jobs as required PR checks.	MODERATE	MEDIUM
H7 — No End-to-End Tests	Add Playwright with Docker Compose; cover Discovery create → start → stream and Connect create → check → alert flows.	HIGH RISK	HIGH
H8 — Assertion-Free Unit Tests	Replace <code>HealthTest</code> and <code>ReachabilityCalculationTest</code> with tests that import <code>App\</code> classes; fail CI on assertion-free unit tests.	HIGH RISK	MEDIUM

5.5 Expected Outcomes

- **Critical paths protected:** Discovery job orchestration, Connect reachability calculation, and MongoDB event streaming have automated regression tests before any service-layer extraction.
- **CI catches full-stack regressions:** Required PHPUnit + Vitest + Playwright gates on every PR prevent untested changes from merging.
- **Contract stability:** JSON Schema tests for 19 API endpoints prevent breaking changes to `{ data: ... }` envelopes and SSE event shapes consumed by the React SPA.
- **Effective coverage above 75%:** Replacing smoke tests and adding Feature/Vitest suites raises measured coverage from ~6% to refactor-safe levels.
- **Parallel runtime drift eliminated:** Contract parity tests (or dev-api removal) ensure local development matches production Laravel behavior.

6. Security Analysis

OVERALL CODEBASE RATING — SECURITY

High Risk

Driven by Critical missing API authentication (H1), High permissive CORS and session stream IDOR (H2), and 9/12 OWASP categories with concrete findings (H5).

EXECUTIVE SUMMARY

Security review covered **backend** (16 PHP application files, 6 API controllers, 19 public Laravel routes), **frontend** (14 TS/TSX/JXS source files), and **dev-api** (6 JS files, 18 Express routes) fetched from `shende-shweta/FSDKC@main` via GitHub REST API and raw content scan. The stack uses Eloquent ORM and the MongoDB PHP driver with parameterized queries — no SQL, NoSQL, or shell injection hotspots were observed. The dominant risk is **complete absence of authentication and authorization** on every API route (Laravel and dev-api), compounded by **wildcard CORS** (`allowed_origins: ['*']` / `cors()` with no origin filter), enabling any website or anonymous client to create jobs, start tests, read transcripts, and import monitors. A **session stream IDOR** lets any caller replay SSE events with only a `session_id` (the route `{id}` is ignored). PHP `extract($request->all())` in

`LegacyReportController` creates variable-scope injection from query parameters. Frontend XSS sinks, client-side secrets, and browser token storage were not observed; `npm audit` reported **zero** CVEs across frontend and dev-api lockfiles. CI runs PHPUnit and frontend build only — no `npm audit`, `composer audit`, or SAST gate. Overall verdict: **High Risk**, driven by one Critical finding (missing API authentication), three High findings (permissive CORS, session IDOR, unauthenticated bulk-import), and 9/12 OWASP categories with concrete findings.

6.1 Security Benchmark Ratings

#	Security KPI	Target	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Critical Vulnerabilities	0	0	1	>1	1 (unauthenticated API surface)	HIGH RISK
H2	High Vulnerabilities	0	<5	5–10	>10	3 (CORS wildcard, session IDOR, bulk-import)	MODERATE
H3	Medium Vulnerabilities	low	<20	20–50	>50	7 (default creds, debug mode, missing headers, no rate limit, exposed DB ports, extract(), FS5 HTTP/CSP)	MODERATE
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	4.2/KLOC (10 findings / 2.4 KLOC)	HIGH RISK
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	25% categories clean (3/12)	HIGH RISK
H6	Critical/High Vulnerable Deps	0	0	1	>1	0 (<code>npm audit</code> critical+high on frontend and dev-api)	GOOD
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	0% flagged (Laravel 12, React 19, Vite 6 — current majors)	GOOD
H8	End-of-Life Dependencies	0	0	1–5	>5	0 EOL majors detected	GOOD

6.5 Actions Required

Finding	Action	Rating	Priority
Missing API Authentication	Add Sanctum/JWT auth middleware to all routes in <code>backend/routes/api.php</code> ; add policy checks in 6 controllers; mirror or remove unauthenticated dev-api.	HIGH RISK	CRITICAL
Permissive CORS	Replace <code>allowed_origins: ['*']</code> in <code>backend/config/cors.php</code> and default <code>cors()</code> in <code>dev-</code>	HIGH RISK	HIGH

	<code>api/src/server.js</code> with env-driven allow-list.		
Session Stream IDOR	Bind <code>session_id</code> to <code>resource_id</code> + user in <code>RealTimeTestService</code> ; validate in <code>StreamController</code> and dev-api <code>streamSession()</code> .	HIGH RISK	HIGH
Unauthenticated Bulk Import	Remove or protect <code>dev-api /bulk-import</code> with auth, schema validation, and rate limiting.	HIGH RISK	HIGH
PHP extract() Variable Injection	Replace <code>extract(\$filters)</code> in <code>LegacyReportController</code> with typed DTO; refactor <code>LegacyDataMapper</code> to explicit key access.	MODERATE	MEDIUM
Default Credentials & Debug Mode	Remove hardcoded passwords from <code>docker-compose.yml</code> and <code>.env.example</code> ; set <code>APP_DEBUG=false</code> outside local.	MODERATE	MEDIUM
Missing Security Headers	Add CSP, HSTS, X-Frame-Options, and X-Content-Type-Options in <code>docker/nginx/default.conf</code> .	MODERATE	MEDIUM
No Rate Limiting	Apply Laravel <code>throttle</code> middleware and <code>express-rate-limit</code> on POST/start endpoints.	MODERATE	MEDIUM
Exposed Database Ports	Remove <code>3306:3306</code> and <code>27017:27017</code> host mappings from <code>docker-compose.yml</code> .	MODERATE	MEDIUM
FS5 — HTTP default & no CSP	Enforce HTTPS <code>VITE_API_URL</code> in production; add CSP to <code>frontend/index.html</code> or nginx.	MODERATE	MEDIUM
DevSecOps — no dependency scan in CI	Add <code>npm audit --audit-level=high</code> and <code>composer audit</code> steps to <code>.github/workflows/ci.yml</code> .	MODERATE	MEDIUM
No Security Audit Logging	Add structured audit log for resource create/start/stream events in Laravel.	MODERATE	MEDIUM

7. Technical Debt

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

High Risk

Driven by High-Risk Code Repository Health (D1), Database Usage (D4), Observability Baseline (D6), and CI/Test Gate for Agent Output (D7).

EXECUTIVE SUMMARY

Analysis covered **87 repository files** from `shende-shweta/FSDKC@main` via GitHub REST API (recursive tree + raw content fetch), including `.github/workflows/ci.yml`, `root/frontend/dev-api` lock files, `backend/composer.json`, Docker Compose stack, manual SQL schema (`docker/mariadb/init.sql`), and five `AGENTS.md` guides. The Klearcom monorepo has a working Docker path and partial CI (PHPUnit + frontend build), but **agentic-harness readiness is High Risk** today. The three most severe gaps are: **(1) missing `backend/composer.lock` and incomplete CI** — no lint/static-analysis gate, dev-api excluded from CI, and no branch-protection signals (`.github/CODEOWNERS`, PR template); **(2) manual flat SQL schema with no Laravel**

migrations — all five MariaDB tables live in one init script with non-idempotent `INSERT` seeds and 80% cross-domain table sharing; **(3) declared-but-unenforced quality tooling** — PHPStan in `backend/composer.json:14-16` has no config file and no CI step, while ESLint/Prettier/pre-commit are absent entirely. AI-assisted development is partially bootstrapped via root and module `AGENTS.md` files plus `docs/CODEBASE_AUDIT_ISSUES.md`, but the parallel Laravel + dev-api runtimes and ~0% effective test coverage mean agents cannot safely verify refactors before merge.

Readiness Benchmark Ratings

#	Dimension	GOOD	MODERATE	HIGH RISK	Measured	Rating
D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	<code>.gitignore</code> present but incomplete; CI in <code>.github/workflows/ci.yml:1-58</code> runs PHPUnit + frontend build only; no <code>backend/composer.lock</code> (404 on fetch); no <code>.github/CODEOWNERS</code> or PR template	HIGH RISK
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused/unmaintained	10/11 security/infra packages wired; <code>phpstan/phpstan</code> declared in <code>backend/composer.json:15</code> but no <code>phpstan.neon</code> and no CI invocation	MODERATE
D3	AI Tool / Agentic Readiness	ready	partial	not ready	5 <code>AGENTS.md</code> files + <code>docs/CODEBASE_AUDIT_ISSUES.md</code> enumerate work; <code>.kiro/</code> minimal; dual Laravel/dev-api runtime and weak CI block safe agent refactors	MODERATE
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	Manual <code>docker/mariadb/init.sql:1-89</code> only — no migrations; FK on 2/4 domain tables; non-idempotent SQL seeds; flat shared schema across Discovery + Connect	HIGH RISK
D5	Development Environment	reproducible	partial	manual / fragile	<code>backend/.env.example</code> + <code>dev-api/.env.example</code> present; no <code>frontend/.env.example</code> ; Docker Compose + Dockerfiles present; TypeScript check via build only — no ESLint/Prettier/pre-commit/PHPStan in CI	MODERATE
D6	Observability / Logging Baseline (additional)	structured logging + health endpoints	partial console logging	no logging baseline	Zero <code>Log::</code> , Monolog, Winston, Pino, or Sentry usage in 50 source files; dev-api uses <code>console.log</code> only (<code>dev-api/src/server.js:273-275</code>)	HIGH RISK
D7	CI / Test Gate for	>80% critical-path	partial gates	no effective gate	CI excludes dev-api; 2 PHPUnit files with 0% effective <code>App\</code> coverage; no	HIGH RISK

	Agent Output (additional)	coverage + lint in CI			frontend tests; no lint/SAST/audit steps in <code>.github/workflows/ci.yml</code>
--	------------------------------	--------------------------	--	--	--

7.8 Actions Required

Gap	Action	Rating	Priority
Missing <code>backend/composer.lock</code>	Run <code>composer update</code> in <code>backend/</code> and commit <code>backend/composer.lock</code> ; add CI step verifying lock file is in sync with <code>composer.json</code> .	HIGH RISK	CRITICAL
Incomplete CI coverage	Extend <code>.github/workflows/ci.yml</code> with dev-api smoke test job, <code>vendor/bin/phpstan analyse</code> , ESLint on frontend, and <code>npm audit --audit-level=high</code> .	HIGH RISK	CRITICAL
No branch protection signals	Add <code>.github/CODEOWNERS</code> and <code>.github/pull_request_template.md</code> ; enable required status checks on <code>main</code> .	HIGH RISK	HIGH
PHPStan declared but unwired	Create <code>backend/phpstan.neon</code> with level 5; add <code>vendor/bin/phpstan analyse</code> to CI; ban <code>extract()</code> via custom rules.	MODERATE	HIGH
Manual SQL schema — no migrations	Generate Laravel migrations from <code>docker/mariadb/init.sql</code> ; restrict <code>init.sql</code> to first-boot seed; add migration step to CI backend job.	HIGH RISK	CRITICAL
Non-idempotent MariaDB seeds	Replace bare <code>INSERT</code> in <code>docker/mariadb/init.sql:71-89</code> with idempotent upserts or guard with <code>INSERT IGNORE</code> .	HIGH RISK	MEDIUM
Shared flat database schema	Assign table ownership per domain; plan schema split documented in <code>init.sql</code> header and module <code>AGENTS.md</code> files.	HIGH RISK	HIGH
Missing <code>frontend/.env.example</code>	Create <code>frontend/.env.example</code> with <code>VITE_API_URL=http://localhost:8080/api</code> ; reference in README quick-start.	MODERATE	LOW
No ESLint / pre-commit enforcement	Add <code>eslint.config.js</code> with React/TS rules; add <code>.pre-commit-config.yaml</code> or enforce via CI only.	MODERATE	MEDIUM
Dual runtime blocks agent isolation	Deprecate dev-api or proxy to Laravel; until then, tag every audit item in <code>CODERBASE_AUDIT_ISSUES.md</code> with runtime scope.	MODERATE	CRITICAL
No observability baseline	Add Laravel <code>Log</code> facade usage in controllers/services; replace dev-api <code>console.log</code> with structured JSON logger; expose <code>/api/health</code> metrics.	HIGH RISK	MEDIUM
No effective test gate for agents	Rewrite PHPUnit tests to import <code>App\</code> classes; add Vitest for frontend hooks; add dev-api route tests; target 80% coverage gate in CI.	HIGH RISK	CRITICAL